For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

k-Nearest Neighbor (k-NN) Queries

Imagine that you're building Zomato, and you've a table called `restaurants` that holds **100 million** entries.

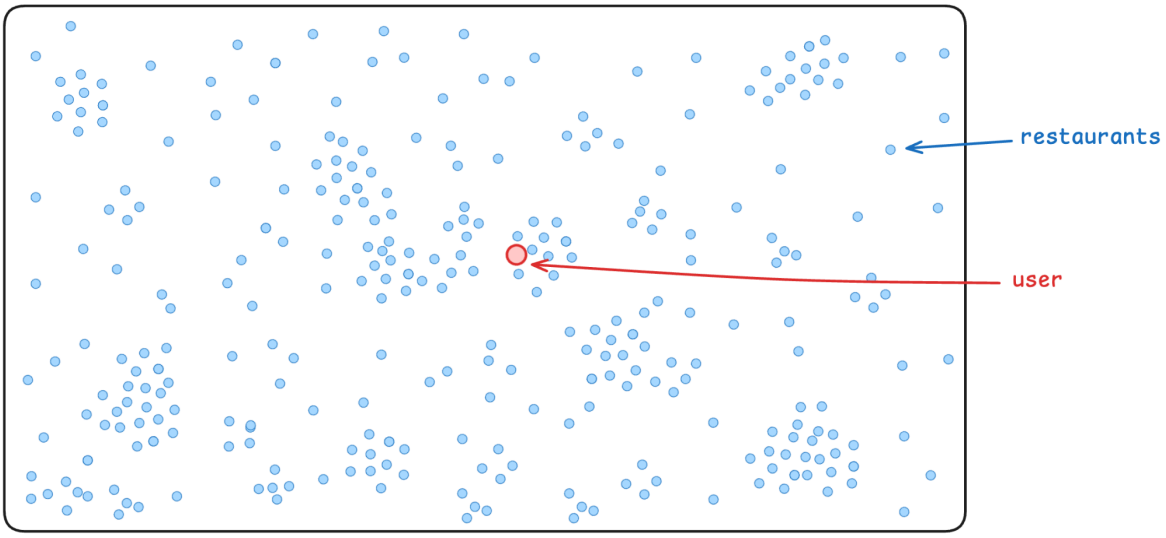
```
create table restaurants(id bigint primary key,
                         name varchar(50),
                         address varchar(200),
                         latitude real,
                         longitude real)
```

Assume that we also have the necessary indexes on the `latitude` and `longitude` columns.

Note: modern SQL databases like postgres support geolocation datatypes via extensions (eg, postgis)

Task

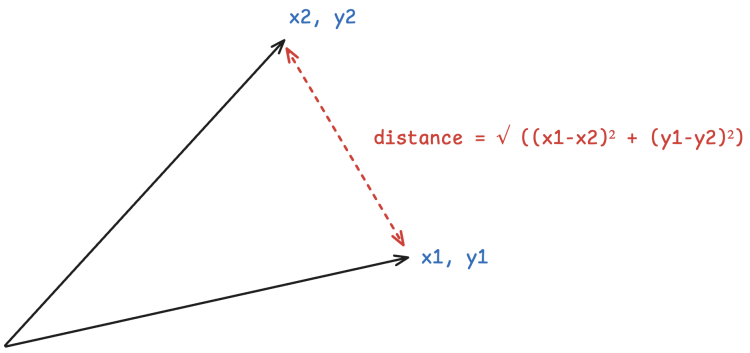
Given a user's geolocation `(latitude, longitude)`, you have to find `k` nearby restaurants.



Attempt 1 - find closest k restaurants

```
select *,
        ((latitude - user_latitude) * (latitude - user_latitude)
         + (longitude - user_longitude) * (longitude - user_longitude)) as  distance
from restaurants
order by distance asc
limit k
```

note: the formula used is the Euclidean Distance (two point distance) formula



Q: What is the time complexity of the above query, assuming that the table has N rows, and we've indexes on the latitude & longitude columns?

O(N log k)

1. Even though we have the indexes, we cannot use them
    a. because we're sorting on the distance, and not the latitude/longitude
    b. this will be a linear scan through the entire table
2. Returning top `k`
    a. `log(k)` for the min/max heap

Given an unsorted array of `n` elements, find the top `k` ⇒ this takes `O(n log k)` time

Attempt 1.1 - find all restaurants within a circle

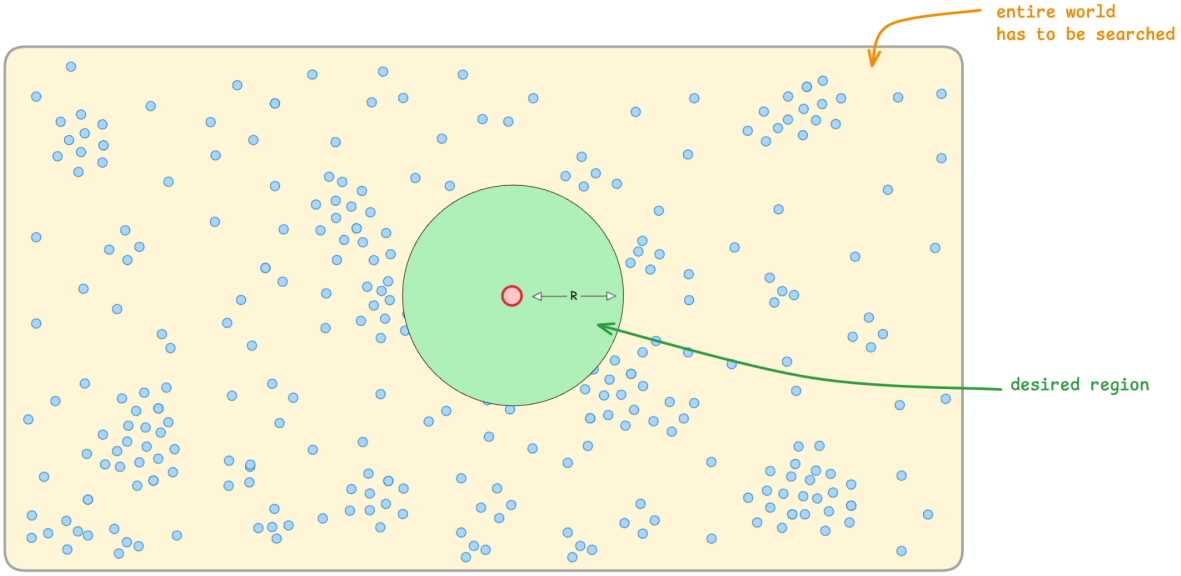
```
select *,
        ((latitude - user_latitude) * (latitude - user_latitude)
          + (longitude - user_longitude) * (longitude - user_longitude)) as distance
from restaurants
where distance <= R
limit k
```

Q: Time Complexity?

`O(N)`

No sorting/heap needed now.
But we still have to perform a linear scan through the entire table – because the distance formula is complex, and the database cannot use the index to optimize this formula.

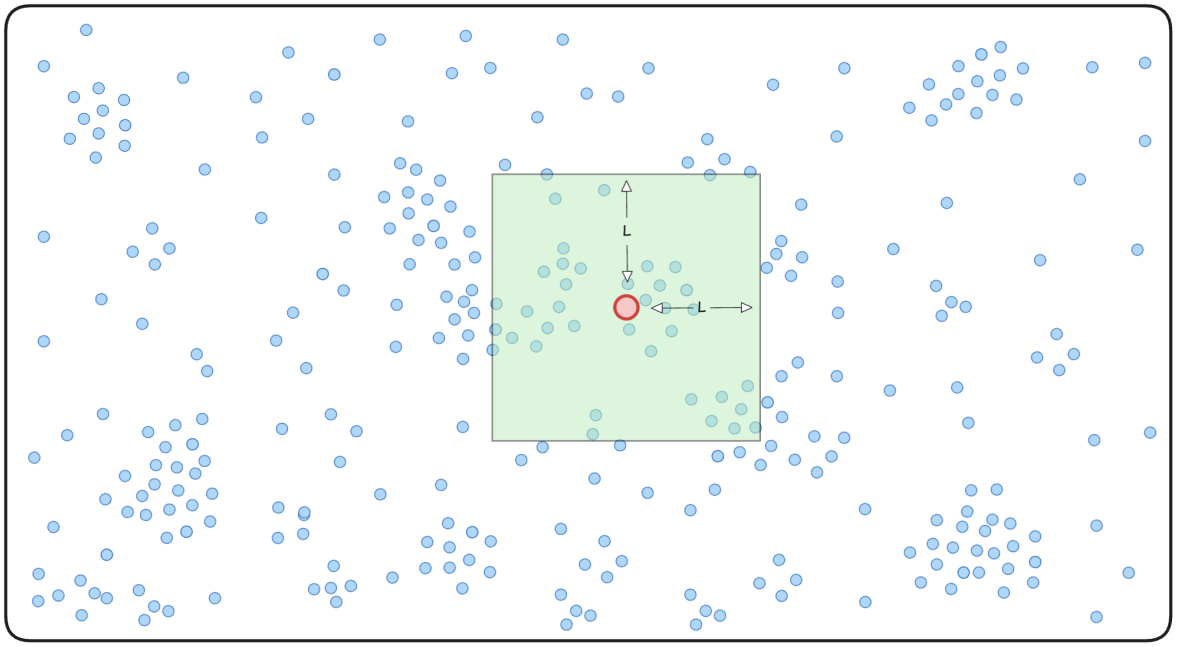
A database index is just a sorted array / BBST
You can do lowerbound/upperbound queries (binary search) using an index.



Circle formulas are complex. Rectangles are simpler!

Attempt 2 - find all restaurants within a square

```
select *
from restaurants
where user_latitude between ( latitude - L) and ( latitude + L)
   and user_longitude between (longitude - L) and (longitude + L)
limit k
```



Q: Time Complexity, assuming that we've index on both `latitude` and `longitude` columns?

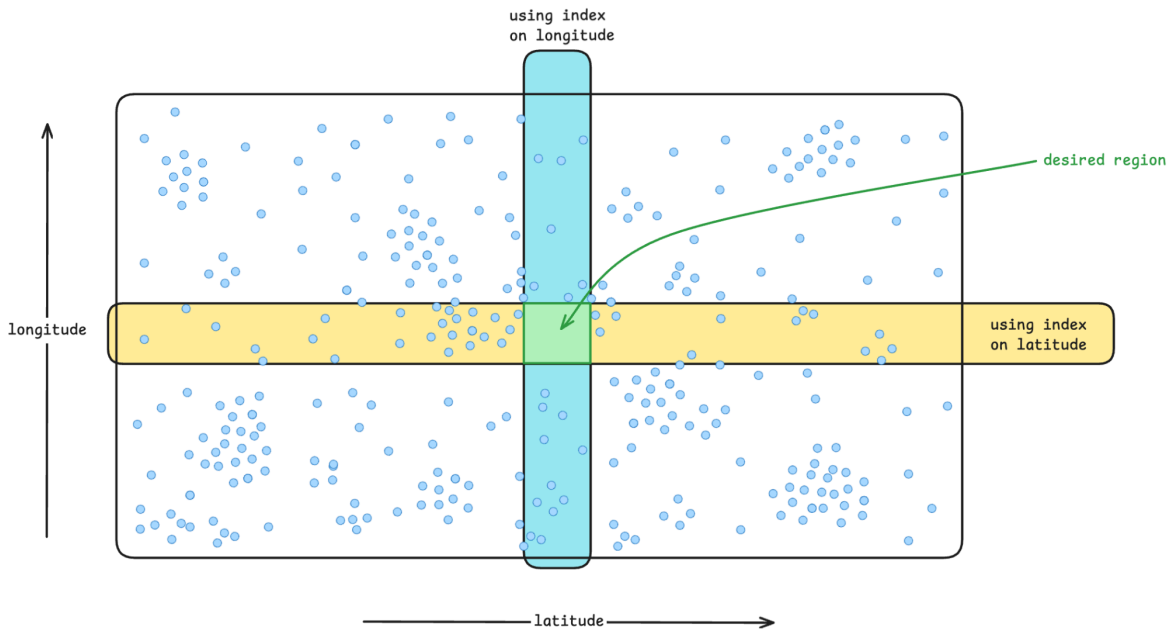
~~`O(log N)`~~

still `O(N)` worst case! Because you can only use 1 index at a time.

1. Database will use the index on latitude to get all the restaurants in the blue region
2. Then, it will use the index to get all the restaurants in the yellow region
3. DB will take intersection of the two sets

We are still searching through a much larger region than what we actually desire!

Note: this query is better than the circle query, but still not perfect!



Wait, really? What if we also had a multi-column indices?

- Single column indices

```
create index idx_lat on restaurants(latitude);
create index idx_lon on restaurants(longitude);
```

- Multicolumn (composite/joint) indices

```
create index idx_lat_lon on restaurants(latitude,
longitude);
create index idx_lon_lat on restaurants(longitude,
latitude);
```

Unfortunately, even after having multi-column indexes, the search will still remain the exact same! *(we will still scan through the blue+yellow region, and then take intersection)*

Why though?

Multi-column indexes can perform range queries only on last column!

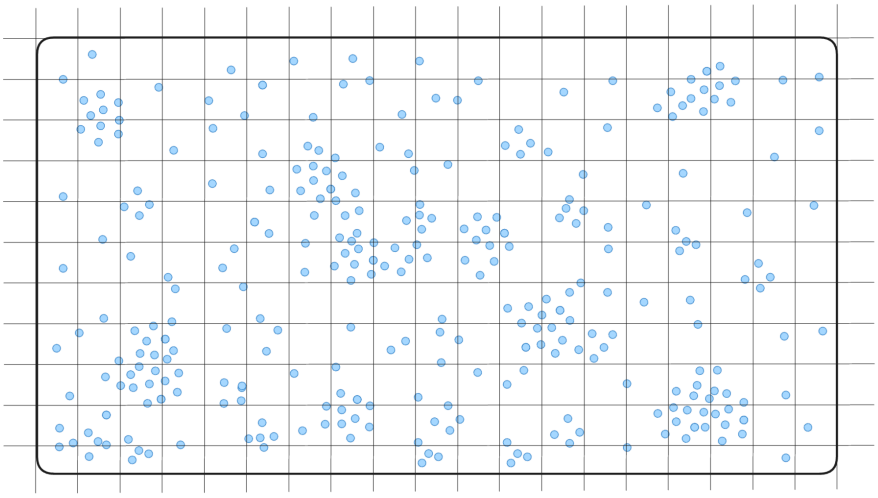
- can do efficiently `O(log N)`
  - exact queries on any columns
    - latitude = X
    - longitude = Y
    - latitude = X and longitude = Y
  - range queries on last column *(exact queries on all previous columns)*
    - latitude = X and longitude between Y1 and Y2

    - range queries on a specific column, and include everything in all the other columns to the right
        - latitude between X1 and X2, and the longitude can be anything
- can NOT do efficiently
    - range queries on both columns
        - latitude between X1 and X2 and longitude between Y1 and Y2

Demo - why Composite Index can't do multi-column range queries

Attempt 3 - fixed grid

1. First break the world into a fixed sized grid



2. For each restaurant, add the `cell_id` in the restaurants table

```
alter table restaurants add cell_id integer;
```

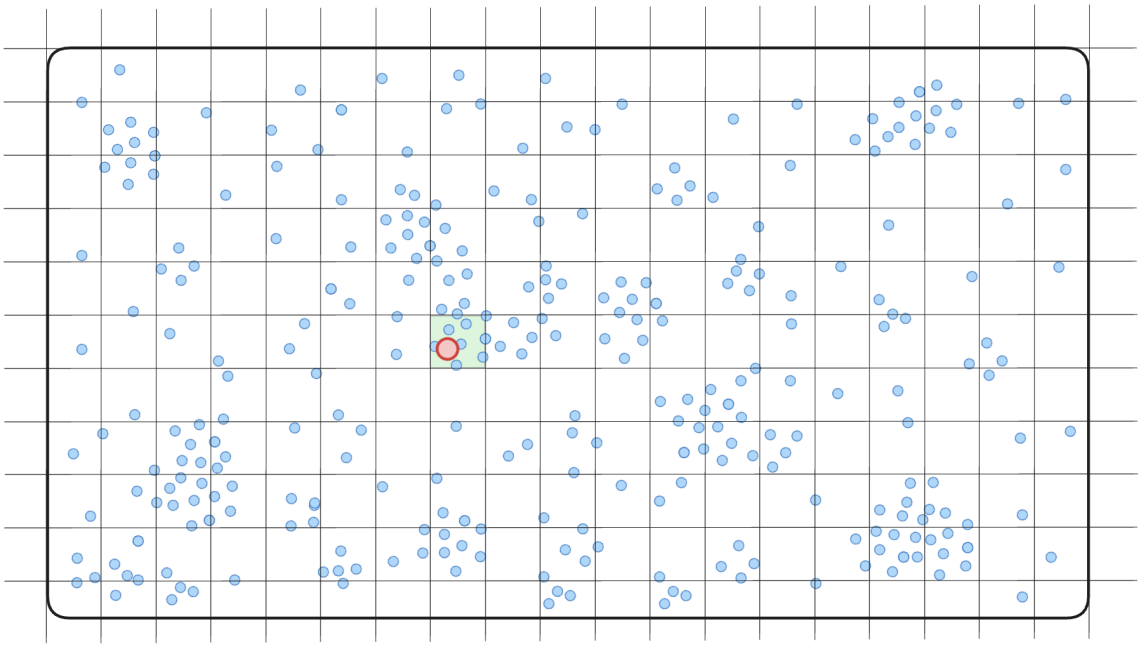
    - we will calculate the `cell_id` of each restaurant based on its geolocation (lat/lon)
    - we will place an index on the `cell_id` column

3. Given the user's geolocation, get the user's cell id

```
get_cell_id(user_latitude, user_longitude)
```

4. Find all restaurants within that cell

```
select *
from restaurants
where cell_id = get_cell_id(user_latitude, user_longitude)
```



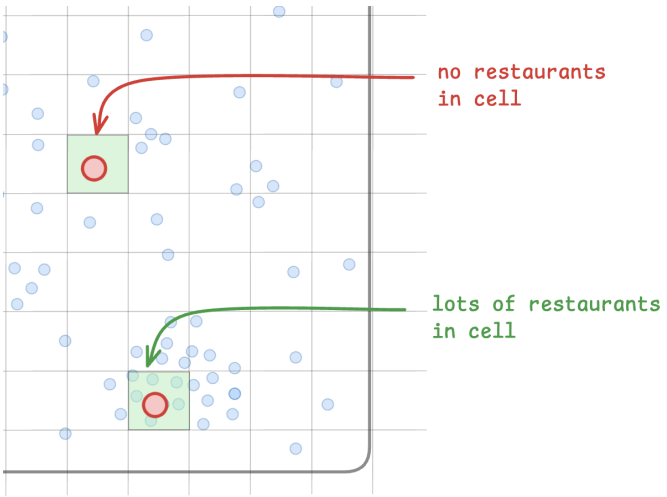
Q: Time Complexity, assuming that we've index on `cell_id`?

finally, this query will achieve our desired complexity!
`O(log N)`

Issue

Density of restaurants changes depending on the location.

Some cells have lots of restaurants, some cells have no restaurants.



So if the user happens to fall in a low density region, we will not be able to find sufficient restaurants for this user.

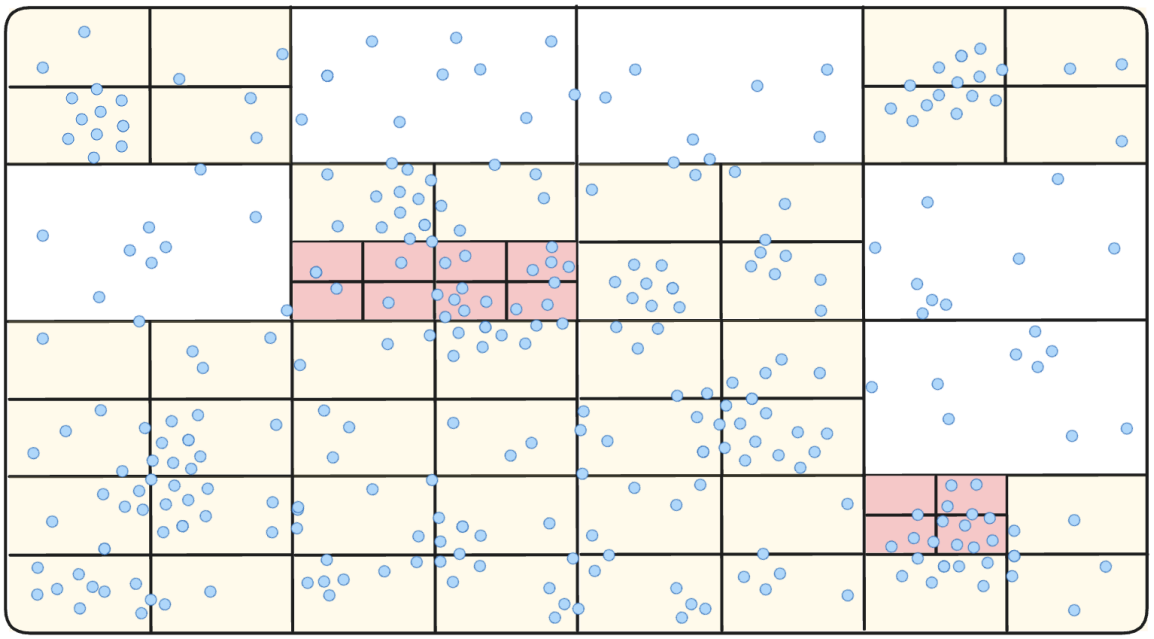
One approach to fixing this could be to simply expand the search to the nearby cells

```
select * from restaurants
where cell_id in ( 18, 17, 19, 5, 6, .. )
```
(surrounding cells)

We don't want a fixed sized grid - we want a **Dynamic Sized Grid!**
We need a grid that considers the density of the region.

QuadTrees

- QuadTrees create a dynamic grid. The grid size is based on the density of the region.
- 2D index: index on two columns where you can do range queries on both the columns! *(Spatial index)*



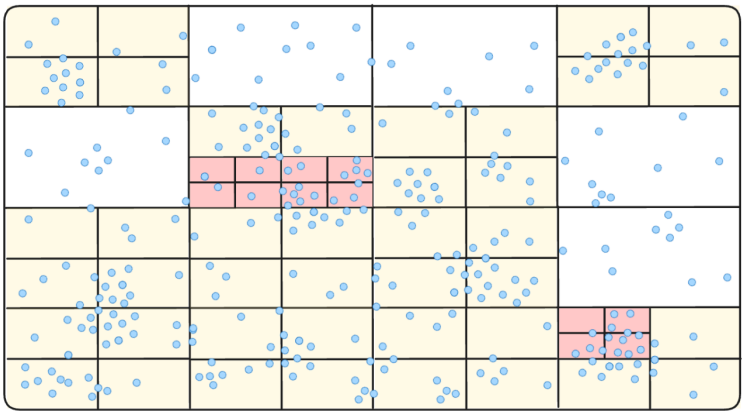
QuadTrees provide efficient range queries on 2 dimensions simultaneously!

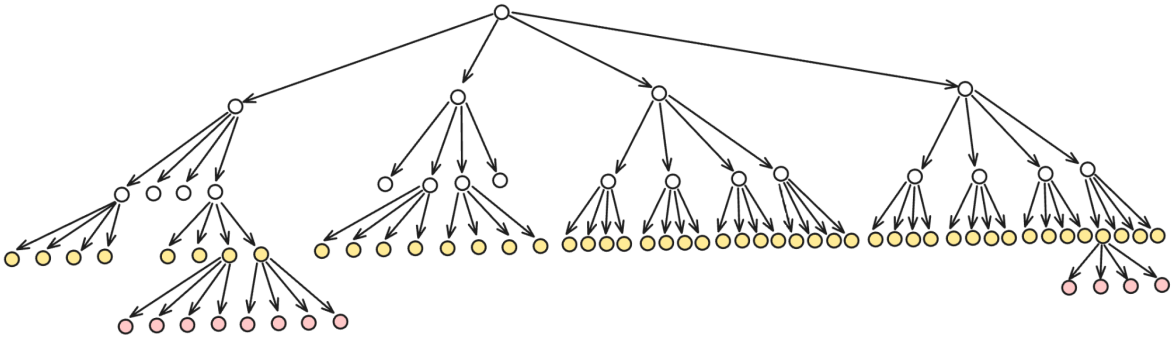
Unlike regular B+tree indexes, which only work in 1 dimension, quad-trees work in 2 dimensions!

Intuition

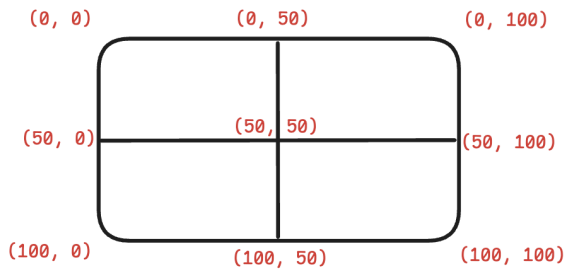
1. Start with all restaurants in the root node
2. If the node contains more than `M=10` restaurants, split the node into 4 equal quadrants
3. Recurse

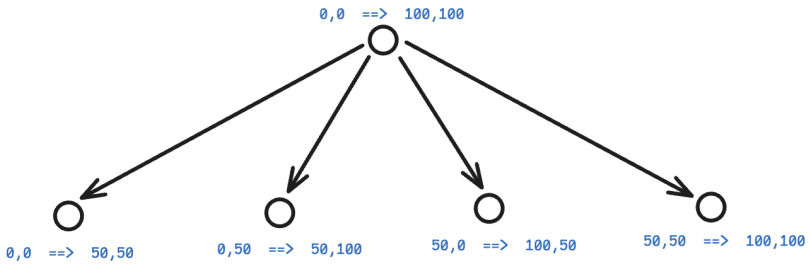
`M` is a configurable parameter





Each node in the tree must be aware of its **coordinates**.





Please practice the QuadTree data structure: [Construct Quad Tree - LeetCode]

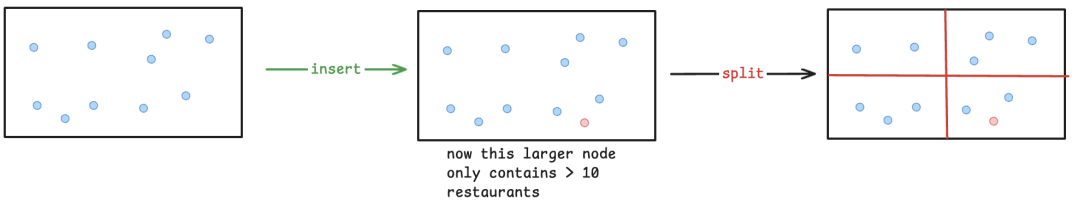
Pseudocode

```
class Restaurant:
    id: int
    latitude: float
    longitude: float
    # the rest of the info will be stored in the db table, and not in the quad tree

class QuadTree:
    id: int
    top, left, bottom, right: (float, float, float, float)

    restaurants: list[Restaurant]  # this will be populated
                                   # only for leaf nodes

    children: list[QuadTree]  # populated only for intermediate nodes

    def find(self,
             latitude: float,
             longitude: float) -> list[Restaurant]:
        """given a user's location, finds the restaurants that
           fall in the same cell as this location"""

        if not ( self.left <= latitude < self.right \
                 and self.top <= longitude < self.bottom ):
            return [] # user's location out of current node's bounds

        if not self.children:  # we're a leaf node
            return self.restaurants

        # recurse over children
        for child in self.children:
```

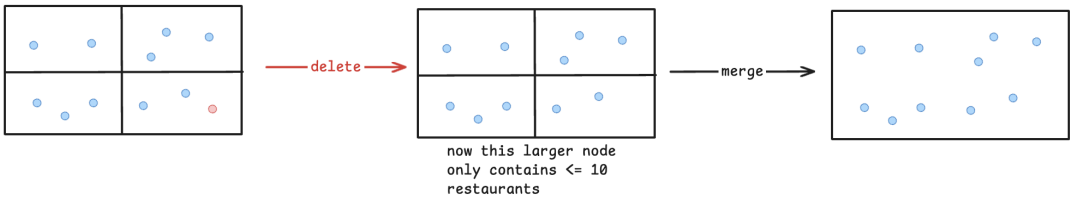
```python
        restaurants = child.find(latitude, longitude)
        if restaurants:
            return restaurants

    def insert(self, restaurant: Restaurant):
        """insert the restaurant"""

        if not ( self.left <= restaurant.latitude <= self.right \
                and self.top <= restaurant.longitude <= self.bottom ):
            return  # location out of current node's bounds

        if not self.children:  # we're a leaf node
            self.restaurants.append(restaurant)

            if len(self.restaurants) > SPLIT_THRESHOLD:
                # create 4 new children
                # with the appropriate coordinates
                # move restaurants to children
                ...
            return

        # recurse over children
        for child in self.children:
            child.insert(restaurant)

    def delete(self, restaurant: Restaurant):
        """delete the restaurant from the tree"""
        # similar to insert
        # find the node
        # if leaf remove restaurant from the list
        #      if parent's count < SPLIT_THRESHOLD, then go to parent
        #      and re-join children
        # otherwise, recurse on our children
```

Note that

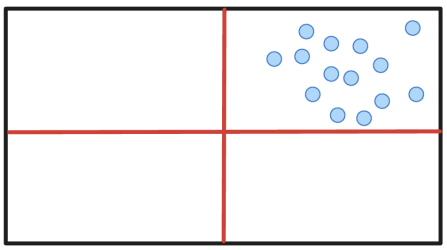
- inserts can cause an existing leaf node to get split



- deletes can cause an existing intermediate node to get merged.

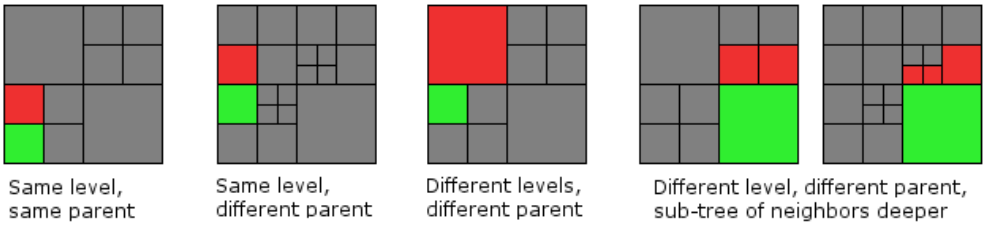


Also note that a leaf node need not always contain `M` restaurants.
It is possible for a leaf node to be empty.



If a user happens to fall in a cell which doesn't contain any restaurants, how will we find the relevant restaurants?

- expand the search to the parent node
  - easy - just go to the parent node, and find all restaurants within it (from all children)
- expand the search to the neighboring nodes
  - much more complex - because neighboring nodes need not come from immediate parent

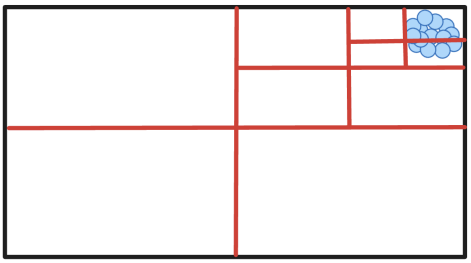


  - however, it can be done efficiently
    - [All Nearest Neighbours via Quadtrees]
    - [Constant Time Neighbor Finding in Quadtrees]
    - [Advanced Octrees 4: finding neighbor nodes | The Infinite Loop]
- If you find that situations like this are happening very frequently, then this simply means that you've a bad value of `M` - try to increase the split threshold

Time Complexity for QuadTree operations

$O(\log_4 N)$ where N is the number of entries in the quadtree
Note that this is the average complexity of insert/delete/find

The worst case can be very bad up to $O(N)$ because the tree can get skewed



Practically, the complexity still remains `O(log₄ N)`

Storing a QuadTree

If we have 100 million restaurants, how large *(in bytes)* will the quadtree be?

```
class Restaurant:
    id: int
    latitude: float
    longitude: float
    # note that other details about the restaurant will not
    # be stored within the quad tree
    # they will be stored in the SQL table

class QuadTree:
    id: int
    top, left, bottom, right: (float, float, float, float)
    restaurants: list[Restaurant]
    children: list[QuadTree]
```

$$\text{Each restaurant takes} \begin{pmatrix} 8b \\ \text{restaurant id} \end{pmatrix} + \begin{pmatrix} 8b \\ \text{latitude} \end{pmatrix} + \begin{pmatrix} 8b \\ \text{longitude} \end{pmatrix} = 24 \text{ bytes of space}$$

$$\text{Each QuadTree node takes} \begin{pmatrix} 8b \\ \text{node id} \end{pmatrix} + \begin{pmatrix} 8b \times 4 \\ \text{children} \end{pmatrix} + \begin{pmatrix} 8b \times 4 \\ \text{coordinates} \end{pmatrix} = 72 \text{ bytes of space}$$

$$\text{Total space} = \frac{24 \text{ bytes}}{\text{restaurant}} \times \#\text{restaurants} \quad + \quad \frac{72 \text{ bytes}}{\text{node}} \times \#\text{nodes}$$

Number of nodes

Only the leaf nodes will store the restaurants. Intermediate nodes do NOT store the restaurants.

(overestimating the number of nodes)
Assume each leaf node stores 1 restaurant on average..

- some leaf nodes have a lot of restaurants – up to `M=10`
- some leaf nodes are completely empty

But we can assume that each leaf has at least `1` restaurant **on average**.

$$
\begin{aligned}
\#\text{ leaf nodes} &= 100\text{ million} \\
\#\text{ parent nodes} &= \frac{100\text{ million}}{4} \quad \because \text{each parent has 4 children} \\
\#\text{ grand-parent nodes} &= \frac{100\text{ million}}{4^2} \\
\#\text{ grand-grand-parent nodes} &= \frac{100\text{ million}}{4^3} \\
&\cdots \\
\#\text{ root nodes} &= 1
\end{aligned}
$$

Total nodes

$$
\begin{aligned}
\#\text{ total nodes} &= 100\text{ million} + \frac{100\text{ million}}{4} + \frac{100\text{ million}}{4^2} + \cdots + 1 \\
&= 100\text{ million} \times \left(1 + \frac{1}{4} + \frac{1}{4^2} + \cdots\right) \\
&= 100\text{ million} \times \frac{4}{3} \\
&\approx 133\text{ million}
\end{aligned}
$$

Total Space

$$
\begin{aligned}
\text{Total space} &= \frac{24\text{ bytes}}{\text{restaurant}} \times \#\text{restaurants} \quad + \quad \frac{72\text{ bytes}}{\text{node}} \times \#\text{nodes} \\
&= \frac{24\text{ bytes}}{\text{restaurant}} \times 100\text{ million restaurants} \quad + \quad \frac{72\text{ bytes}}{\text{node}} \times 133\text{ million nodes} \\
&= 2.4\text{ GB} + 9.6\text{ GB} \\
&= 11\text{ GB}
\end{aligned}
$$

We will need around `11 GB` to store a quadtree with 100 million restaurants

Do we need Sharding?

Absolutely not!
`11 GB` is actually small enough to fit in the RAM

Typical production servers can have up to 64GB of RAM easily..

A single server can hold the entire quad tree to store 100 million nodes.
Because the complexity of operations is $O(\log_4 N)$ and all operations are happening purely in the RAM, the single server will be able to handle 10,000 - 100,000 operations / second!

What about durability? What happens if the quadtree server crashes?

The quadtree is not storing any additional info - every piece of info that it needs is already present in the SQL db.

If the server crashes, we can just spin p another QuadTree server and rebuild the tree from scratch
Rebuilding the entire tree will be an $O(N \log_4 N)$ operation - it will only take a few seconds at max to rebuild.

$$N = 100M$$
$$N \log_4 N = 100M * 13.xyz$$

$$2^{10} = 1024 \sim 1000 = 10^3$$
$$2^{10} \sim 10^3 \text{ (1K)}$$
$$2^{20} \sim 10^6 \text{ (1M)}$$
$$2^{26} \sim 64M$$
$$2^{27} \sim 128M$$
$$\log_2(100M) \sim 26.xyz$$
$$\log_4(100M) \sim 13.xyz$$

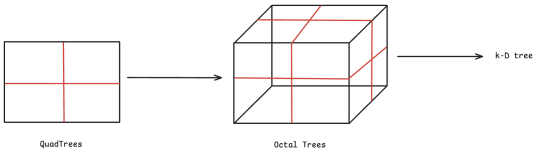
Mandatory Assignment (DSA)

[Construct Quad Tree - LeetCode]

Resources on QuadTrees (optional)

1. **Theory:** [CMU notes - quadtrees.pdf]
2. **Visualization**
    - [An interactive explanation of quadtrees.]
    - [Quadtree Visualizer]

3. **Generalizations (k-d trees):** k-d tree - Wikipedia



4. **Other uses (collision detection / simulations):**
    - 2D Collisions with Quadtree test
    - Quadtrees and The Barnes-Hut Algorithm: The Making of a Gravity Simulation
5. **Proximity services:** FAANG System Design Interview: Design A Location Based Servi…

GeoHash

1. **Explanation**
    - Geohash: the algorithm inside and out - Part 1
    - Geohash - Wikipedia
2. **Code:** Geohash encoding/decoding
3. **Converter:** Geohash Converter
4. **Support in MySQL:** Spatial Geohash Functions
5. **Proximity services:** FAANG System Design Interview: Design A Location Based Servi…

8.37 => 8.50

Doubts

How do you retain something that you've learnt? How do you tackle forgetfulness?

Only solution → Revision
Every now and then you need to keep revising the content.

Revision comes in multiple ways

1. Practice
    a. deliberate: practice for the sake of practice
    eg. practicing DSA problems
    b. on-the-job: if you're frequently working on complex problems that require DSA solutions, then you will automatically end up practicing DSA on a daily basis
2. Explicit revision
    a. boring – resolved via detailed & high quality notes
    b. time consuming – **Spaced Repetition**

Spaced Repetition via Flashcards

Magic pill for all your woes!

Imagine if there's a software which claims

1. insert some piece of knowledge in the software
2. every day you come and revise
3. the software will guarantee that you will **NEVER** forget this piece of knowledge!

- Every major learning platform - KhanAcademy, DuoLingo, Brilliant.org .. all of them use spaced repetition
- Ivy league law & medical colleges - it is mandatory to use spaced repetition - it is part of the curriculum
- Memory competitions - winners of Jeopardy ..

Anki - Anki is a flashcard program that helps you spend more time on challenging material, and less on what you already know.

Making a Second Brain - obsidian / notion